TANSZÉKVEZETŐ

DIPLOMATERVEZÉSI FELADAT

Vértesaljai Bálint (KL3N7K)
Mérnökinformatikus hallgató részére

Gaussian splatting integrációja Godot környezetben

Napjainkban igen komoly fejlődésen mentek keresztül az inverz grafikai módszerek, amelyek a képalkotás folyamatát megfordítva végeznek 3D rekonstrukciót képi információk alapján. Ezek között kiemelten nagy figyelmet kapott a 3D Gaussian Splatting (3DGS), amely a színteret egy speciális, gaussi normális eloszlásokkal kiegészített pontfelhőként reprezentált radiancia mező formájában rekonstruálja. Ezen reprezentáció renderelése igen hatékonyan elvégezhető, a klasszikus 3D modellek raszterezéséhez hasonló módon. A 3DGS reprezentáció igen széles körben kezd elterjedni, már ipari szabványok is kezdenek megjelenni [2].

Ezen korszerű technológia egy különösen érdekes alkalmazása grafikus assetek előállítása számítógépes játékokhoz, pl. a népszerű Godot engine-ben. Ehhez egyrészt szükséges a 3DGS reprezentáció integrációja a grafikai és fizikai motorba, másrészt a szokványos 3D modellekkel való interakció megvalósítása is.

Hivatkozások:

[1] Kerbl et al.: "3D Gaussian splatting for real-time radiance field rendering." ACM Trans. Graph. 42.4 (2023): 139-1.

[2] https://www.khronos.org/news/press/gltf-gaussian-splatting-press-release

A hallgató feladatai:

- Ismerkedjen meg az inverz grafika és a 3DGS technológia alapjaival.
- Végezzen irodalomkutatást a 3DGS-hez kapcsolódóan.
- Integrálja a 3DGS reprezentáció megjelenítését a Godot játékmotorba.
- Végezze el a 3DGS reprezentáció integrálását a Godot fizikai rendszerébe, biztosítsa a megfelelő interakciót a hagyományos 3D modellekkel.
- Implementálja szabványos, tömörített 3DGS formátumok importálását [2].
- Vizsgálja meg az instancing alkalmazási lehetőségeit 3DGS modellekre.
- Vizsgálja meg 3DGS modellek újra-világítható megjelenítésének lehetőségeit.

Tanszéki konzulens: Dr. Vaitkus Márton

Budapest, 2026. március 20.

Dr. Kiss Bálint
egyetemi docens
tanszékvezető

Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Irányítástechnika és Informatika Tanszék

H-1117 Budapest, Magyar Tudósok krt. 2. IE444.
Tel: +36 1 4632057, Fax: +36 1 4634112
www.iit.bme.hu • e-mail: bkiss@iit.bme.hu
e-mail: vaitkus@iit.bme.hu

Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Irányítástechnika és Informatika Tanszék

Gaussian splatting integrációja Godot környezetben

Diplomaterv

Készítette
Vértesaljai Bálint

Konzulens
Dr. Vaitkus Márton

2026. 08. 27.

Tartalomjegyzék

Kivonat **6**

Abstract **7**

1. Bevezetés **8**

1.1. Inverz renderelés 8

1.2. Gaussian Splatting 8

1.3. Godot 8

1.4. Gaussian Splatting a Godot-ban 8

1.5. Motiváció 9

1.6. A diplomaterv további szerkezete 9

2. Irodalomkutatás **10**

2.1. Gaussian Splatting témájú cikkek 10

2.1.1. 3D Gaussian Splatting for Real-Time Radiance Field Rendering 10

2.1.2. Relightolhatóság 10

2.1.3. Időbeliség 10

2.1.4. Szabványos GLTF formátum 10

2.1.5. Generatív AI és inverz módszerek a képszintézisben 10

2.2. Más játékmotorok, Gaussian Splatting integrációjuk 11

2.3. Gaussian Splatting implementációk Godot-ban 11

2.3.1. GodotGaussianSplatting by 2Retr0 11

2.3.2. godot-gaussian-splatting by haztro 11

2.3.3. godot-gaussian-splatting ReconWorldLab 11

2.4. Választott módszer, technológia 11

3. Önálló munka bemutatása **12**

3.1. Önlab 1 alatt végzett megoldás felvázolása 12

3.1.1. Egyszerre több pontfelhő 13

3.1.2. Pontfelhők transzformációja 14

3.1.3. Automata objektum-megtalálás 16

3.1.4. Mélységhelyes összekombinálás 17

3.1.5. Projektek kombinálása 18
3.1.6. Adatok beolvasása 19
3.2. Dipterv 1 alatt végzett megoldás felvázolása 20
3.2.1. RenderedImage pull vs SplatMesh push 20
3.2.2. Elsőosztályú polgárok 21
3.2.3. AMD Bug 22
3.2.4. CompositorEffect 22
3.2.5. Addon-ok uniózása 23
3.2.6. Igazi dinamikus láthatóság 23
3.2.7. Instancing 23
3.2.8. Relightolhatóság 24
3.2.8.1. Saját fényszámítással 25
3.2.8.2. A Godot fényszámításával 26
3.3. Kitérő: Typst 28
3.3.1. Rövid bemutatás 29
3.3.2. Miért jobb 29

4. Önálló munka értékelése, eredmények 30
4.1. When Hamsters Attack TD 30
4.2. Tanszék modelljének példafelhasználása 30
4.3. Asset library plugin 30
4.4. Instancing mérése felhasznált memóriával 30
4.5. Még mit lehetne értékelésnek, eredménynek? 30
4.6. További teendők a Dipterv 2-ig (végleges doksiig) 30
4.7. TODO a dipterv1 doksi leadásáig! 31

5. Összefoglaló 32

Irodalomjegyzék 33

Függelék 34

HALLGATÓI NYILATKOZAT

Alulírott *Vértesaljai Bálint*, szigorló hallgató kijelentem, hogy ezt a diplomatervet meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem. A Függelékben egyértelműen megjelöltem, hogy a mesterséges intelligencia eszközeit alkalmaztam-e a dolgozat elkészítéséhez; amennyiben igen, annak módját és mértékét a táblázatban közöltem. Tudomásul veszem, hogy a mesterséges intelligenciával generált tartalomért – annak mértékétől függetlenül – teljes felelősséggel tartozom.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző(k), cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövege pedig a BME Címtárban regisztrált személyek számára elérhető legyen. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Budapest, 2026. 08. 27.

Vértesaljai Bálint
hallgató

Kivonat

Napjainkban igen komoly fejlődésen mentek keresztül az inverz grafikai módszerek, amelyek a képalkotás folyamatát megfordítva végeznek 3D rekonstrukciót képi információk alapján. Ezek között kiemelten nagy figyelmet kapott a 3D Gaussian Splatting (3DGS), amely a színteret egy speciális, gaussi normális eloszlásokkal kiegészített pontfelhőként reprezentált radiancia mező formájában rekonstruálja. Ezen reprezentáció renderelése igen hatékonyan elvégezhető, a klasszikus 3D modellek raszterezéséhez hasonló módon. A 3DGS reprezentáció igen széles körben kezd elterjedni, már ipari szabványok is kezdenek megjelenni.

Ezen korszerű technológia egy különösen érdekes alkalmazása grafikus assetek előállítása számítógépes játékokhoz, pl. a népszerű Godot engine-ben. Ehhez egyrészt szükséges a 3DGS reprezentáció integrációja a grafikai és fizikai motorba, másrészt a szokványos 3D modellekkel való interakció megvalósítása is.

Abstract

Nowadays, inverse graphics methods have undergone a very serious development, which perform 3D reconstruction based on image information by reversing the process of image creation. Among them, 3D Gaussian Splatting (3DGS) has received a lot of attention, which reconstructs the scene in the form of a radiance field represented as a special point cloud supplemented with Gaussian normal distributions. The rendering of this representation can be performed very efficiently, in a way similar to the rasterization of classical 3D models. 3DGS representation is becoming very widespread, and industry standards are already starting to appear.

A particularly interesting application of this modern technology is the production of graphic assets for computer games, e.g. in the popular Godot engine. This requires, on the one hand, the integration of the 3DGS representation into the graphics and physics engine, and on the other hand, the implementation of interaction with conventional 3D models.

1. Fejezet

Bevezetés

1.1. Inverz renderelés

Inverz renderelésnek azt nevezzük, amikor képekből állítunk elő 3D modelleket. Lehet például egy adott objektum körbefotózva sok szögből, vagy egy labor körbevideózva. Ez egy bonyolult folyamat, és sokféle módszer létezik rá.

1.2. Gaussian Splatting

A Gaussian Splatting egy inverz rendering módszer, aminél a jelenetet gausszi eloszlású átlátszósággal rendelkező 3D foltokkal próbáljuk a lehető legjobban leírni. Képekből előállítani a modellt elég erőforrásigényes, viszont a modellek megjelenítése megfelelő minőség esetén kifejezetten közel is tud lenni a valósághoz. Nagy előnye, hogy a megjelenítőt sima, standard grafikus pipeline-nal is lehet implementálni, és valós időben, 100-as nagyságrendű FPS-sel renderelhető.

1.3. Godot

A Godot Engine egy játékmotor, nyílt forráskódú, és az elmúlt években jelentős növekedésnek örvendett. Megtalálhatóak benne a legfontosabb eszközök egy játék fejlesztéséhez, viszont a teljes motor egy nagyon könnyű, lightweight megvalósításnak örvend: a motor egyetlen futtatható fájllal futtatható, installációt nem igényel, és a teljes mérete 200MB alatt van.

A 4-es verzióban a következő generációs renderer Vulkan alapú, és ennek bővítésére, és a Vulkan-hoz történő viszonylag alacsony szintű hozzáférésre is különféle lehetőségeket nyújt.

1.4. Gaussian Splatting a Godot-ban

Alapból a Godot rendererje(i) sima, háromszöghálós modellek megjelenítésére optimalizált, a Gaussian Splatting nincs a hivatalosan támogatott feature-ök között. Található viszont egy Godot-ban implementált Gaussian Splatting megjelenítő az interneten.

1.5. Motiváció

Amennyiben az engine-be jobban integrálnánk gaussian splatting modelleket, képesek lehetnénk őket nem csak megjeleníteni, hanem a háromszöghálós modellek mellett, akár egyszerre, használni őket játékok készítésére.

1.6. A diplomaterv további szerkezete

A továbbiakban bemutatom a témában megtalálható forrásokat, implementációkat, és leírom az általam elvégzett munka tervét, majd a megvalósítását, és a megvalósítás értékelését. Végül összegzem a leírtakat.

2. Fejezet

Irodalomkutatás

2.1. Gaussian Splatting témájú cikkek

2.1.1. 3D Gaussian Splatting for Real-Time Radiance Field Rendering

A Gaussian Splattinget az [1] cikk vezette be, ezzel nagy sikert aratva. Az általuk készített megoldás a képekből történő paraméteroptimalizációra, amivel a modelleket (pontfelhők paramétereit) lehetett elkészíteni, kifejezetten jól lett megírva, dicsérték.

2.1.2. Relightolhatóság

A relightolhatóság game engine-ek esetén egy különösen fontos dolog, mivel ezzel lehet előre nem meghatározott mozgás esetén is életszerűbb megvilágítással ellátni a pontfelhőket. A [2] forrásban megjelölt program segít ilyen, újralightolható pontfelhőket generálni háromszöghálós modellekből. A diplomaterv során az újralightolhatóságot ilyen modelleken tesztelem.

2.1.3. Időbeliség

Külön érdekesség lehet, ha időben is változik a felvett jelenet, ilyenkor a pontfelhő időbeli transzformációját is el lehet tárolni. Erről szól a TODO cikk.

2.1.4. Szabványos GLTF formátum

Az adatokat valamilyen módon el is kell tárolni, erre sokféle megoldás született. Különféle kulcsszavakkal rendelkező `.ply` fájlok, `.splat` fájlok, és egyéb fájformátumok is használatban vannak. Nem rég jött ki a témában egy GLTF szabvány [3] is, amit a Khronos Group kezel.

2.1.5. Generatív AI és inverz módszerek a képszintézisben

A témában jól összegyűjtött információk forrása lehet többek között a BME-n nemrég indult szabadon választható, Generatív AI és inverz módszerek a képszintézisben című tárgy honlapja és diasorai, ami a [4]-nél érhetők el.

2.2. Más játékmotorok, Gaussian Splatting integrációjuk

Érdemes lehet utánanézni, hogy a Unity, és Unreal játékmotorok hogyan állnak a témához, gaussian splatting modellek megjelenítésére, és egy játékba történő integrálására milyen lehetőségek vannak. Mennyire támogatott az eredeti kiadók (Unity, Epic) által, illetve milyen plugin-ok / addon-ok elérhetőek, és ezek milyen minőségűek.

Én azért a Godot mellett döntöttem, mivel Free / Open Source filozófiájával, és minimalista méretével, szemléletével ő állt hozzám a legközelebb. Az integrációt valószínűsíthetően mindegyik engine esetén el fogja valaki végezni.

2.3. Gaussian Splatting implementációk Godot-ban

Mivel az engine, és a Gaussian Splatting téma is népszerű, ezért már találhatóak hasonló témájú implementációk.

2.3.1. GodotGaussianSplatting by 2Retr0

[5] volt az eredeti implementáció, ami egy Gaussian Splatting modellek megtekintésére szolgáló megjelenítő. Ez egy modell megjelenítését és körüljárását támogatta.

2.3.2. godot-gaussian-splatting by haztro

Ez egy másik implementáció, amelyik a projekt kezdetekor alulmaradt a fentebbitől. Nemrég kibővítette a készítője, célszerű lenne megint kipróbálni. [6]

2.3.3. godot-gaussian-splatting ReconWorldLab

Ez [7] az implementáció már a diplomaterv írása közben bukkant fel. Ez már, az előzőekkel szemben, képes több modellt megjeleníteni, és transzformációt applikálni rájuk. Ez az implementáció a 2.3.1. fejezetben bemutatott implementáción alapul. Mivel a céljaink nagyjából megegyeznek, úgy döntöttem, felveszem vele a kapcsolatot, és kollaborációt kezdeményezek. Ezt véltem a megfelelő megoldásnak, mivel így nem végezzük el kétszer ugyanazt a munkát, és mindketten tudjuk használni az eredményeket.

2.4. Választott módszer, technológia

A választott módszer a 2.3.1-ben leírt projekt kibővítése, illetve a [7] repository kiegészítése, a feladatlapban leírtak implementálásával.

3. Fejezet

Önálló munka bemutatása

3.1. Önlab 1 alatt végzett megoldás felvázolása

Az önlab 1-es GitHub repository-m itt [8] érhető el, a kóddal és a dokumentációval együtt.

Adott a megtekintő program, amely be tud tölteni egy `.ply` kiterjesztésű splat-os modellt, és meg tudja azt jeleníteni a játékmotor keretein belül. Illetve adott példaként az autó-szimuláló program. Adottak továbbá splat-os modellek, amiket lehet használni.

Ennek megfelelően a féléves munka a következő részegységekre osztható:

- Megjeleníteni egyszerre több pontfelhőt
- Ezeket a pontfelhőket függetlenül mozgatni
- Feltölteni pontfelhőnként transzformációs mátrixokat, hogy tetszőlegesen mozoghassanak a modellek
- Ezeket a transzformációkat megfelelően feltölteni minden képkockában, lehetőleg automatizált módon megtalálva a hozzájuk tartozó játékobjektumot, ahonnan a transzformáció származik
- Mélységhelyesen összekombinálni a pontfelhős, és a sima modelleket
- Összevonni a két projektet (gsplat-os, illetve az autós)
- Robusztusabbá tenni a `.ply` fájl-beolvasást, hogy a `.splat`-ból [SuperSplat] (https://superspl.at/editor)-tal konvertált `.ply` fájlokat is képes legyen beolvasni
- Javítani felmerülő problémákat
  - Stuttering: camera és transzformációs mátrix feltöltésének a process priority állítása lett a megoldás
  - Artifactos mélység: a shader kód párhuzamosan tölt be adatokat, és dolgozza fel őket, ehhez viszont szükséges shared bufferek használata

3.1.1. Egyszerre több pontfelhő

A megjelenítő egy pontfelhő megjelenítésére volt felkészítve, nekünk viszont egy játékban jórészt több pontfelhőt is célszerű megjeleníteni: például az autó teste és a 4 kereke már eleve 5 külön objektum. Ehhez azt találtam ki, hogy a CPU-n összeuniózom a pontfelhőket, az adataikat, viszont elmentem azt is, hogy mely indexeknél vannak a határok. Ezután a GPU-ra feltöltéskor kihasználok egy float-ot, ami eddig padding célt szolgált, és minden feltöltött splat esetén egy ID-t is továbbítok, ami megmondja, hogy hányadik objektumhoz tartozik.

```
class_name PlyFile extends Resource

var size : int
var vertices : PackedFloat32Array
var properties : Array[StringName]
var split : Array[int] # indices where new objects start

...

static func merge(pc1 : PlyFile, pc2 : PlyFile) -> PlyFile:
  var merged := PlyFile.new()
  merged.size = pc1.size + pc2.size
  assert(pc1.properties.hash() == pc2.properties.hash())
  merged.properties = pc1.properties
  merged.vertices = PackedFloat32Array(pc1.vertices)
  merged.vertices.append_array(pc2.vertices)
  merged.split.append_array(pc1.split)
  merged.split.append(pc1.vertices.size())
  for s in pc2.split:
    merged.split.append(s + pc1.vertices.size())
  return merged

...

static func load_gaussian_splats(point_cloud : PlyFile, stride : int,
    device : RenderingDevice, buffer : RID, should_terminate_reference :
Array[bool],
    num_points_loaded : Array[int], callback : Callable):
...
      ### Opacity
      points[b+6+4] = 1.0 / (1.0 + exp(-p[v+54]))
```

```
      ### ID for differenciating between objects
      points[b+11] = 0
      for k in point_cloud.split:
        if v >= k:
          points[b+11] += 1
...
```

3.1.2. Pontfelhők transzformációja

Miután a GPU-n már lehet tudni, hogy melyik splat melyik objektumhoz tartozik, így lehetséges objektumonként más transzformációt alkalmazni a splatokra. Ehhez létre kellett hozni, és fel kell tölteni egy új uniform buffert:

```
const MAX_OBJECT_COUNT := 16 # number of gsplat object transforms, same
as in gsplat_projection.glsl
var object_transforms : Array[Transform3D]
...

  descriptors['transforms'] =
context.create_uniform_buffer(16*MAX_OBJECT_COUNT*4)

...

  context.device.buffer_update(descriptors['transforms'].rid,
        0, 16*MAX_OBJECT_COUNT*4, get_transforms())

...

func update_object_transforms(transforms: Array[Transform3D]) -> void:
  object_transforms = transforms

func get_transforms() -> PackedByteArray:
  var fbuf := PackedFloat32Array()
  assert(len(object_transforms) <= MAX_OBJECT_COUNT)
  var t := []
  for i in object_transforms:
    t.append(Projection(i))
  for i in (MAX_OBJECT_COUNT - len(object_transforms)):
    t.append(Projection.IDENTITY)
  for i in MAX_OBJECT_COUNT:
    fbuf.append_array([ t[i].x[0], t[i].x[1], -t[i].x[2], -t[i].x[3],
              t[i].y[0], t[i].y[1], -t[i].y[2], -t[i].y[3],
              -t[i].z[0], -t[i].z[1], t[i].z[2], t[i].z[3],
```

```
                -t[i].w[0], -t[i].w[1], t[i].w[2], t[i].w[3]])
  var bytebuf := PackedByteArray()
  bytebuf.resize(4 * fbuf.size())
  bytebuf.fill(0)
  for i in range(len(fbuf)):
    bytebuf.encode_float(i*4, fbuf[i])
  return bytebuf
```

Amit pedig a shaderben fel tudunk használni:

```
// same as in gaussian_splatting_rasterizer.gd
#define MAX_OBJECT_COUNT 16

...

struct Splat {
  vec3 position;
  float time;
  float covariance[6]; // Contains top triangle of symmetric matrix
  float opacity;
  float id;
  float sh_coefficients[16*3]; // Spherical harmonic coefficients in
increasing order
};

...

layout (std140, set = 0, binding = 7) restrict uniform Transforms {
  mat4 transforms[MAX_OBJECT_COUNT];
};

...

  // --- FRUSTUM CULLING ---
  vec3 splat_pos = splat.position*model_scale;
  vec4 view_pos = view_matrix * transforms[int(splat.id + 0.5)] *
vec4(splat_pos, 1.0);
  vec4 clip_pos = projection_matrix * view_pos;

  ...

  mat3 curr_transform = mat3(transforms[int(splat.id + 0.5)]);
```

```
  mat3 cov_mx = curr_transform * DECODE_COVARIANCE(splat.covariance) *
transpose(curr_transform);
  const vec3 covariance = project_covariance(cov_mx, splat_scale,
view_pos.xyz, dims);

...
```

3.1.3. Automata objektum-megtalálás

Jelenleg nem támogatott új objektumok létrehozása illetve eltüntetése, láthatóvá illetve láthatatlanná tétele futásidőben. Támogatott viszont az, hogy egy, a Godot Node-ok között is megtalálható objektumot a SceneTree-hez hozzáadva automatán be legyen töltve a splat-os objektum, és a megfelelő Node transzformációja legyen rá hattatva, figyelembe véve a szülő node-ok transzformációját is, így lehetséges valós időben mozgatni, transzformálni őket ugyanúgy, mint a sima modelleket.

A következő script a `class_name` segítségével meg fog jelenni az editorban, mint hozzáadható Node, és lesz egy állítható paramétere, a `ply_file`. Ide be lehet állítani a megfelelő path-t.

```
class_name SplatMesh extends Node3D

@export var ply_file: String

func is_splat_mesh():
  pass
```

Ezután a gyökérnode induláskor összeszedi az objektumokat, és eltárolja őket, hogy a transzformációikat el tudja küldeni:

```
var splat_meshes : Array[SplatMesh] = []

func _ready() -> void:
  find_by_method(self, StringName("is_splat_mesh"), splat_meshes)
  assert(len(splat_meshes) <=
GaussianSplattingRasterizer.MAX_OBJECT_COUNT)
  var splat_filenames := []
  for m in splat_meshes:
    splat_filenames.append(m.ply_file)

  init_rasterizer(splat_filenames)
    ...
```

```
# source: https://forum.godotengine.org/t/how-do-you-get-all-nodes-of-a-certain-class/9143
func find_by_method(node: Node, method_name : StringName, result : Array) -> void:
  if node.has_method(method_name) and node.is_visible_in_tree():
    result.push_back(node)
  for child in node.get_children():
    find_by_method(child, method_name, result)
```

Az összeszedéshez biztosan van jobb módszer, mint a tagfüggvénynév alapján való keresés, de én limitált keresés során nem találtam, és ez működik. Típuslekérdezés csak beépített típusokra ad eltérő eredményt.

```
func _process(delta: float) -> void:
    ...
  var splat_transforms : Array[Transform3D] = []
  for m in splat_meshes:
    splat_transforms.append(m.global_transform)
  rasterizer.update_object_transforms(splat_transforms)
```

Fontos megemlíteni a szálakat, ugyanis a rasterizer függvényeit alapvetően a render thread-en hívjuk, ez az objektum-transzformációk feltöltése viszont, a kameratranszformációk lekérdezéséhez hasonlóan a „sima” thread-en történik.

3.1.4. Mélységhelyes összekombinálás

Ahhoz, hogy a képen össze lehessen fésülni a pontfelhős, és a sima megjelenítést, amelyek teljesen külön pipeline-on futnak, szükséges a pontfelhős megjelenítésnél is, a végső renderelt képen egy pixelenkénti mélységérték szerzése. Ennek segítségével legalább pixelenként el lehet dönteni, hogy a két renderelt kép közül melyik nyerjen. A sima pipeline-nál már eléri a játékmotor a mélységet, nekünk a splat-os megjelenítésnél kell ezt elérhetővé tennünk:

Projection:

```
  data.pos_z = (ndc_pos.z + 1.0)*0.5;
```

Render:

```
shared float[WORKGROUP_SIZE] pos_z_tile;

...
```

```
    for (uint i = 0; i < num_iterations && shared_t > MIN_FACTOR; ++i) {
    ...
        for (uint j = 0; j < chunk_size && t > MIN_ALPHA; ++j) {
            ...
            float pos_z = pos_z_tile[j];
            ...
            if (alpha > 0.2) {
                weighted_depth += (1.0 - pos_z) * alpha * t;
                total_weight += alpha * t;
            }
            ...
            t *= (1.0 - alpha);
        }
        ...
    }
    vec3 heatmap_color = mix(vec3(0,0,1), vec3(1,0.2,0.2),
num_splats*5e-4) * (1.0 - t) * heatmap_factor;
    float final_depth = total_weight > 0.0 ? (weighted_depth /
total_weight) : 0.0;
  imageStore(rasterized_image, ivec2(image_pos), vec4(blended_color +
heatmap_color, final_depth));
  ...
```

Main spatial shader:

```
void fragment() {
  ALBEDO = srgb_to_linear(texture(render_texture, SCREEN_UV).rgb);
  DEPTH = texture(render_texture, SCREEN_UV).a;
}
```

3.1.5. Projektek kombinálása

A splat-os és az autós Godot projektek összefésülése egészen könnyen ment, a fájlok összemásolásán túl a `project.godot` fájlt kellett kompatibilissá tenni mindkét projekt részére, többek között a definiált (billentyűzet) bemenetet, és a globális scripteknek megfelelő scripteket (autoload) kellett átemelni. Szerencsére a `project.godot` egy szöveges fájlformátum, sima config fájlként viselkedik, így akár kézzel is könnyű volt az egyes bejegyzéseket átemelni, a maradék beállítást pedig könnyen megtaláltam az editorban.

Továbbá mindkét projektben volt az alapértelmezett jelenet, aminek a node-jaihoz a szükséges scriptek hozzá voltak adva. Kis logikázás után úgy láttam, hogy a splat-os projekt gyökérnode-ja kell, hogy maradjon, többek között azért is,

hogy a hozzáadott SplatMesh-eket megtalálja a jelenetben. Nem volt különösebb gond az autós projekttel, ha annak a gyökérnode-ját a splat-os projekt gyökérnode-ja gyerekeként tettem be.

Ami következményként nehézséget okozott, az a stuttering kiküszöbölése, amit az okozott, hogy az autós projektben a kamerát egy script állította, míg a splatos projektben ugyanezen (aktív) kamera transzformációját egy script továbbította a GPU-nak, és ez a két script valamiért nem meghatározott sorrendben futott. A process priority állításával a probléma megoldódott.

3.1.6. Adatok beolvasása

A bemeneti adatformátum a `.ply`, illetve ennek egy speciális esete. Eredetileg a megjelenítő projekt csak olyan modelleket tudott megjeleníteni, amiknek a 62 property-je mind szerepel, és egy konkrét sorrendben van:

```
x, y, z, nx, ny, nz, f_dc_0, f_dc_1, f_dc_2, f_rest_0, f_rest_1,
f_rest_2, f_rest_3, f_rest_4, f_rest_5, f_rest_6, f_rest_7, f_rest_8,
f_rest_9, f_rest_10, f_rest_11, f_rest_12, f_rest_13, f_rest_14,
f_rest_15, f_rest_16, f_rest_17, f_rest_18, f_rest_19, f_rest_20,
f_rest_21, f_rest_22, f_rest_23, f_rest_24, f_rest_25, f_rest_26,
f_rest_27, f_rest_28, f_rest_29, f_rest_30, f_rest_31, f_rest_32,
f_rest_33, f_rest_34, f_rest_35, f_rest_36, f_rest_37, f_rest_38,
f_rest_39, f_rest_40, f_rest_41, f_rest_42, f_rest_43, f_rest_44,
opacity, scale_0, scale_1, scale_2, rot_0, rot_1, rot_2, rot_3
```

Ez akkor ütközött problémába, amikor egy másik forrásból származó modellt szerettem volna megjeleníteni, ami `.splat` formátumban volt. Addig nem probléma, hogy az online [Supersplat](https://superspl.at/editor) segítségével át lehet konvertálni ezt `.ply` fájllá, viszont ebből a modellből hiányoztak bizonyos property-k, 14 volt összesen, és a meglevők sem megfelelő sorrendben voltak:

```
x, y, z, opacity, rot_0, rot_1, rot_2, rot_3, f_dc_0, f_dc_1, f_dc_2,
scale_0, scale_1, scale_2
```

Így módosítottam a beolvasó részt, hogy 0-ként olvassa a hiányzó adatokat, illetve a megfelelő pozícióba tegye a property-ket. Ez azért működik, mert ami hiányzik, az a 45 db spherical harmonikusokhoz szükséges érték, amiket lehet 0-ra inicializálni, illetve a normálvektor, amit pedig nem használ fel a beolvasó.

Példareferencia egy kódrészletre: a parse függvény az 1. kódrészleten látható.

```
func parse(path : String) -> void:
  var file := FileAccess.open(path, FileAccess.READ)
  var line := file.get_line().split(' ')
  while not line[0] == 'end_header':
    line = file.get_line().split(' ')
    match line[0]:
      'format':   file.big_endian = line[1] == 'binary_big_endian'
      'element':  size = int(line[2])
      'property': properties.push_back(line[2])
  vertices = file.get_buffer(size*len(properties) * 4).to_float32_array()
  if properties.hash() != DEFAULT_PROPERTIES.hash():
    var prop_inverse := {}
    for i in properties.size():
      prop_inverse[properties[i]] = i
    var new_vertices := PackedFloat32Array()
    new_vertices.resize(size*DEFAULT_PROP_CNT)
    for i in size:
      for pi in DEFAULT_PROP_CNT:
        new_vertices[i * DEFAULT_PROP_CNT + pi] =
                    vertices[i * len(properties) +
prop_inverse[DEFAULT_PROPERTIES[pi]]]
                    if DEFAULT_PROPERTIES[pi] in prop_inverse else 0
    properties = DEFAULT_PROPERTIES.duplicate()
    vertices = new_vertices
```

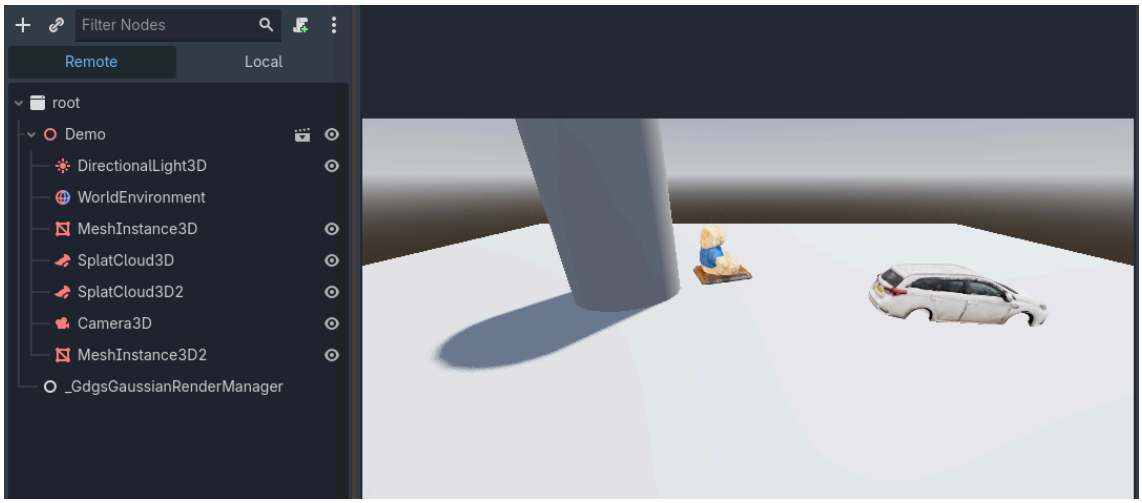
1. kódrészlet: Példa egy kódrészlet számozására

3.2. Dipterv 1 alatt végzett megoldás felvázolása

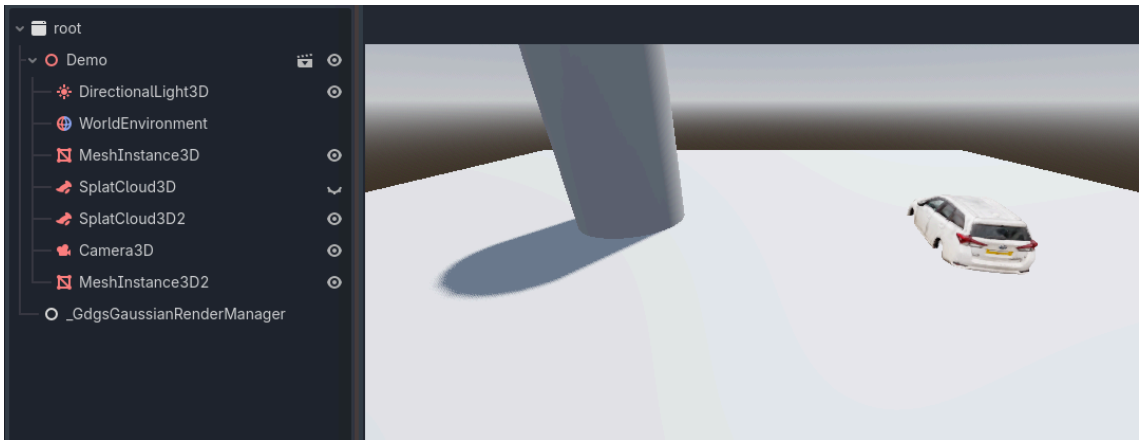
3.2.1. RenderedImage pull vs SplatMesh push

Az eredeti megoldás szerint a RenderedImage válogatta össze a jelenetből a kirajzolandó pontfelhő node-okat, és a transzformjaikat. Ezzel az új megoldással a pontfelhő node-ok maguk regisztrálhatnak, és jelentkezhetnek le a kirajzolásról.

Ez azt is könnyűvé tette, hogy amennyiben a node láthatósága változik, akkor leregisztráljon, és ezzel eltűnjön ténylegesen a jelenetből. Ez része annak a törekvésnek, hogy a pontfelhők az editorba, illetve a játékba is jobban legyenek integrálva, azaz az editorba épített feature-ök (mint pl. a láthatóság) úgy működjenek rajtuk, mint ahogy a felhasználók várják. A nem látható pontfelhő az 1. ábrához képest a 2. ábrán láthatóan nem látható.



1. ábra: Látható a plüssmaci



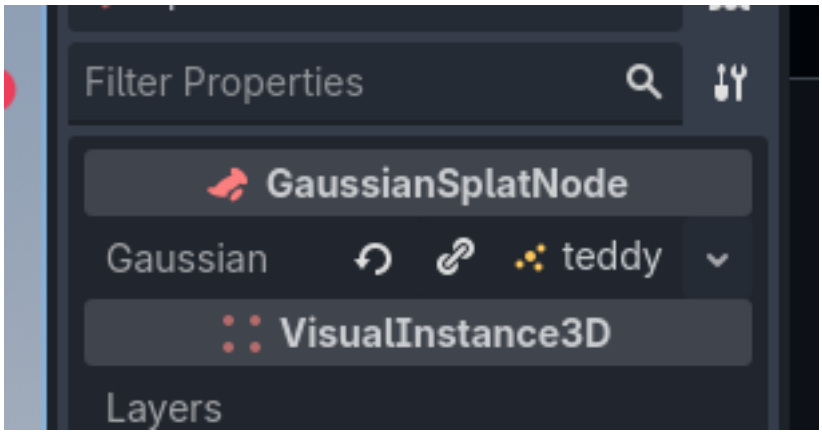
2. ábra: Nem látható a plüssmaci

Hozzá kell tenni, hogy ez a megoldás még nem volt a legcélravezetőbb: minden láthatóságváltoztatáskor a teljes éppen aktuálisan látható splatos adatot feltöltöttük a GPU-ra. Ez a 3.2.6. fejezetben került javításra.

3.2.2. Elsőosztályú polgárok

További editorintegrációhoz egyéb apróságokat is elvégeztem, ilyen a 3. ábrán látható ikonok készítése a splat-os node-okhoz, illetve resource-okhoz, a betöltés és a tárolt adat szétcsatolása, és ezzel az editor automatikusan be is tudja tölteni az eddig számára ismeretlen file-t egy Godot resource-á. Ezzel például exportált buildekhez is automatikusan hozzá tudja tenni ezt a fájlt, és ott már nem kell a betöltést lefuttatnia.

Az 1. táblázatnál láthatóak a node-hoz, illetve a resource-hoz társított ikonok szöveges leírásai is. Ezzel tudom tesztelni a Typst táblázat captionjeit, illetve referenciáit.



3. ábra: A Node és a Resource saját ikonokat kaptak

GaussianSplatNode	GaussianResource
Piros, átfedő pöttyök	Sárga, kisebb sugarú pöttyök

1. táblázat: Táblázat teszt

3.2.3. AMD Bug

A tesztelések során figyelmes lettem egy AMD hardveren történő bugra, ezt a VulkanRadixSort könyvtárra vezettem vissza, és az eltérő subgroup méret okozhatta. A hibát én is kijavítottam, és láttam azt is, hogy upstream is kijavították már, csak az általam használt eredeti megjelenítő nem frissítette a dependency-t. Ezt jeleztem az ő repositoryjukban is, a megfelelő fixet megjelölve [9]. A linkelt issue-ban találhatóak a további linkek a kapcsolódó fixekre.

3.2.4. CompositorEffect

Godot 4.4-től már lehetőség van a rendering pipeline-ba jobban belenyúlni, CompositorEffect-eket írva. Az eddigi, eredeti megjelenítő egy QuadMesh-t jelenített meg egy MeshInstance3D-vel, ami lényegében mindig az igazi kamera előtt lebegett, így tudta a pontfelhőket megjeleníteni. Én még önlab alatt megcsináltam azt hozzá, hogy az átlátszóság alapján összeollózza az eredeti, sima raszteres kamera által adott képpel, ez viszont több helyen látszott, hogy egy elég összetákolt megoldás volt. Manuálisan kellett meghívni a saját renderelésünket, de csak akkor hogyha tényleg szükségünk volt a képre, különben elkezdett szaggatni az editor. A használt textúrák felszabadításának működése nem világos, illetve ablak átméretezésekor is le kell kezeljünk speciális dolgokat, ami nem egyértelmű, hogy teljes körűen sikerül.

3.2.5. Addon-ok uniózása

Ezek miatt felkerült a teendők listájára a CompositorEffect-es megoldásnak a vizsgálata is. Közben a félév során egy kínai implementáció is előkerült itt: [7]. Mérlegelve a szituációt arra jutottam, hogy nem érdemes mindkettőnknek teljesen külön dolgozni a saját programján, így átfutottam az ő kódját, és mivel mindkét projekt (és az a projekt is, amelyet mindketten használtunk: [9]) MIT licensz alatt elérhető, unióztam a fontosabb feature-jeinket. A kommunikáció felvételekor leírtam 1-2 dolgot, ennek eredménye lett, hogy ő is kijavította az AMD bugot, illetve valamilyen kovarianciás matekot, amitől a splatok kiterjedései is jól lettek transzformálva, illetve ezután készítettem én is neki egy PR-t az editor ikonokkal, illetve dinamikus láthatósággal és instancinggel. Ezt ő mergeelte is, és megtagelt az acknowledgementsben is.

Én pedig az ő implementációja alapján kipróbáltam a CompositorEffect-es megoldást, és tényleg sok problémát egyből megoldott. Így az uniózott projekt már azt használja a továbbiakban.

3.2.6. Igazi dinamikus láthatóság

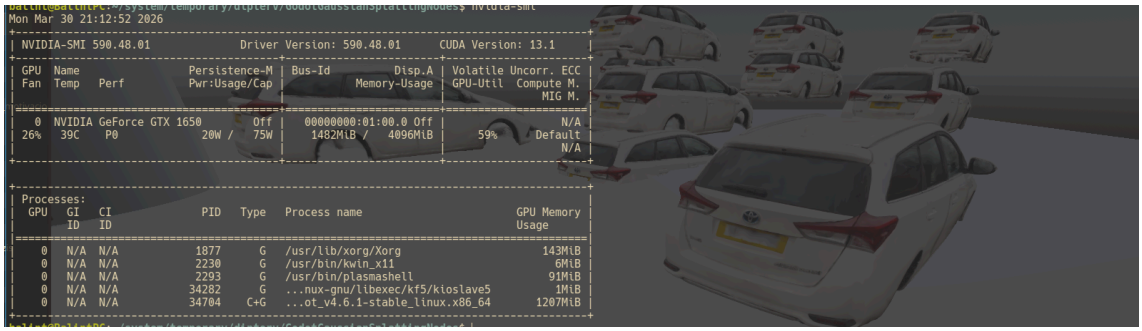
A PR készítése közben a korábbi dinamikus láthatóság problémájából kiindulva biztosan úgy akartam újból megoldani a problémát, hogy láthatóság változtatásakor ne legyen szükség az összes splat adat újbóli feltöltésére. Ez elég könnyen meg is volt oldható úgy, hogy a transzformáció mellé (ami amúgy is minden mozdulatnál újra feltöltődik) pakoltam egy igaz-hamis értéket, ami alapján a shader el tudja dobni a hozzá tartozó splatokat, ezzel megvalósítva a nemláthatóságot.

3.2.7. Instancing

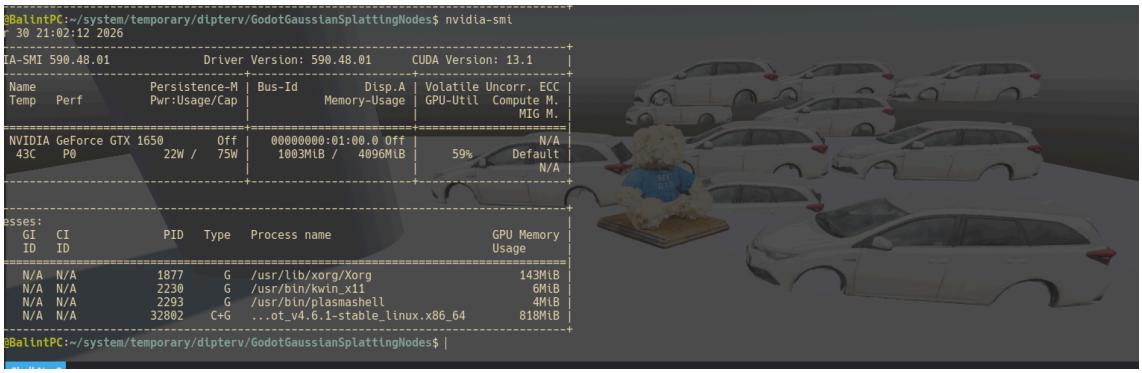
Hogyha már itt voltam a feltöltésnél, úgy láttam, hogy a feladatkiírásban is szereplő instancinget is akkor a legmegfelelőbb implementálni. Ennél a splat-os modelleket kell különválasztani az objektumoktól.

Eddig minden splat egy extra ID-t kapott, és az ID-edik transzformációs mátrixszal lett eltranszformálva, így megoldva azt, hogy az egyes objektumok máshogyan mozogjanak. Az optimalizációs lehetőség ott jön be, hogyha 1 db kocsi helyett 10 db van a jelenetben, akkor a hozzá tartozó százezer-millió nagyságrendű splat adatait nem szükséges 10-szer tárolni a GPU-n, hiszen ezek ugyanazok.

A megoldás pedig nem nagyon nehéz: az ID buffer legyen kétszer akkora, és minden minden párban az egyik ID mondja meg a transzformációs mátrixot, a másik ID pedig a splatot. Így lehet egy modellnek több transzformációs mátrixa is, csak az ID bufferben kell kétszer benne lennie az összes splat ID-jének. Ez az `nvidia-smi` parancs használatával láthatóan jelentősen javította a memóriahasználatot: a 4. ábrán 1207MiB-t használt az instancing előtti állapot, míg az 5. ábrán 818MiB-t, a képeken látható 10-es nagyságrendű autómennyiséggel.



4. ábra: Instancing előtti memóriahasználat



5. ábra: Instancing utáni memóriahasználat

3.2.8. Relightolhatóság

A relightolhatóság egy komplex téma, de egy játékmotor esetén nagyon fontos. Alapból a gaussian splates modellek gyakran rendelkeznek gömbi harmonikusokkal kódolt nézeti irány-függő színekkel, splatonként. Ezzel lehet egyszerű tükröződéseket, csillanásokat szimulálni, és a modell tud színt változtatni annak függvényében, hogy melyik irányból nézzük.

Ezzel szemben egy relightolható / újraszínezhető modell nem nézeti irányfüggő színeket tárol magában splatonként, hanem valamilyen fajta anyagtulajdonságokat, például PBR (roughness, metallic, normal stb.) értékeket. Ennek segítségével nem csak a nézeti iránytól függhet a szín, hanem a teljes, komplex jelenettől, kiemelt hangsúlyt fektetve a fényforrás(ok)ra. Ez egy játékban kifeje-

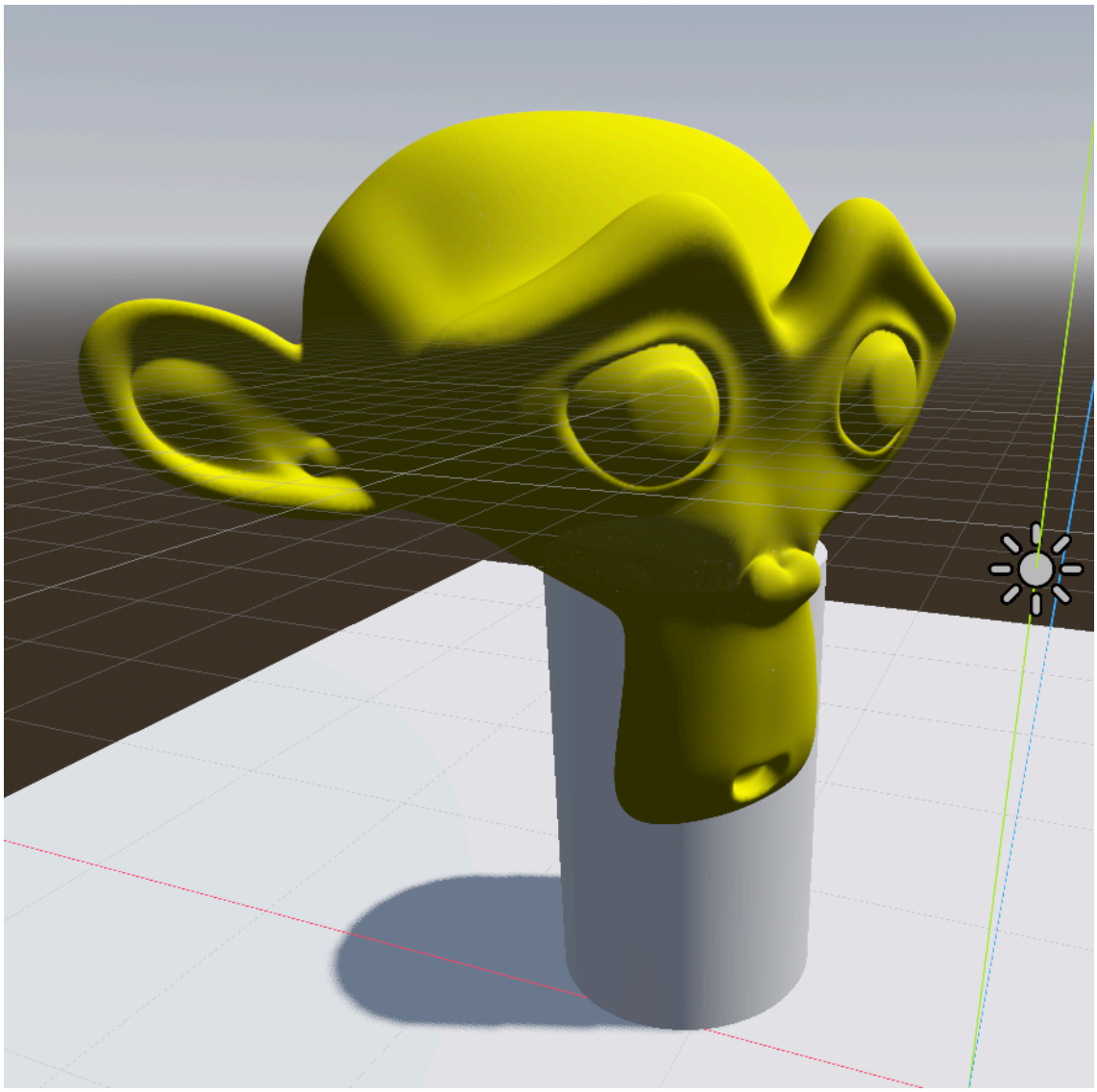
zetten fontos, például ha egy splat-es modell irányt változtat, akkor a nap másik oldalát süti. Ennek megfelelően a színei is teljesen megváltoznak, amit leginkább ilyen módszerekkel lehet megvalósítani.

A lehetőségek felmérésekor többféle megoldás is célravezetőnek tűnt, mindkettő a saját erősségeivel és gyengeségeivel. Végül arra jutottam, hogy mindkettőt megvalósítom, összehasonlítási céllal.

3.2.8.1. Saját fényszámítással

Az első lehetőség az, hogy mi adjuk át manuálisan a beszámítandó fényforrásokat, és mi számoljuk a színeket a megadott paraméterekből. A prototípust úgy csináltam, hogy egy irányfényforrást tudunk átadni. Ennek a hátránya a manuális fényforrás-összegyűjtés, illetve a shading eltérő lehet a többi, engine által shadelt mesh-étől. Hatalmas előnye viszont, hogy működik a 3.2.4. fejezetben tagalt CompositorEffect-es megoldással.

A 6. ábrán is látható, hogy a megvilágítás működik, de a 7. ábrához képest azért alulmarad a többi megvilágítással való egyezőségben. Ezen az ambient light állításával némiképp szerintem lehetne segíteni, de ugyanolyan jó sosem lesz.



6. ábra: Suzanne, a Blender majom megvilágítva általunk

3.2.8.2. A Godot fényszámításával

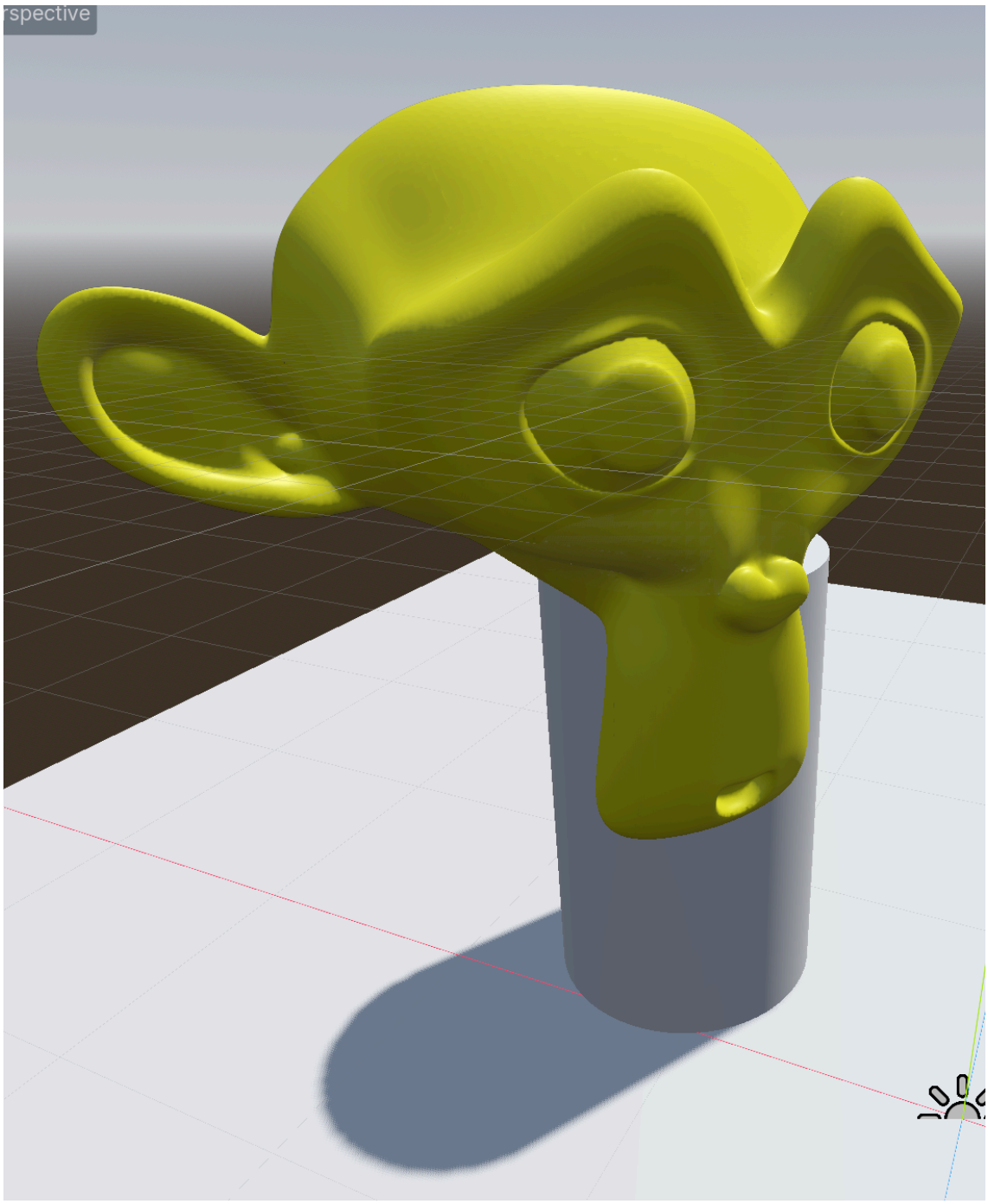
A Godot fényszámításának használatakor viszont elő kellett venni az eredeti, Fullscreen quad-os megoldást, mivel a Godot-nak ezen kereszül tudom átadni a PBR értékeket (egy textúrában), ami alapján ő fogja elvégezni a relightolást, a belül tárolt, gyorsítótárazott, stb. fények alapján. Így ez automatikusan tudna működni pontfényforrásokra és spotlightokra is (csak még egy bug miatt nem tud, lásd 4.6), amint azt a 8. ábra is mutatja.

Még egy nagyon komoly hátránya ennek a megoldásnak, hogy ez leginkább egyáltalán nem tud félig átlátszó splat-okat kezelni (ami a gaussian splattingnél egy nagyon fontos tulajdonság), mivel a Godot-nak pixelenként csak egy színt, és PBR material tulajdonságokat lehet átadni. Az pedig nem túlzottan megoldható,

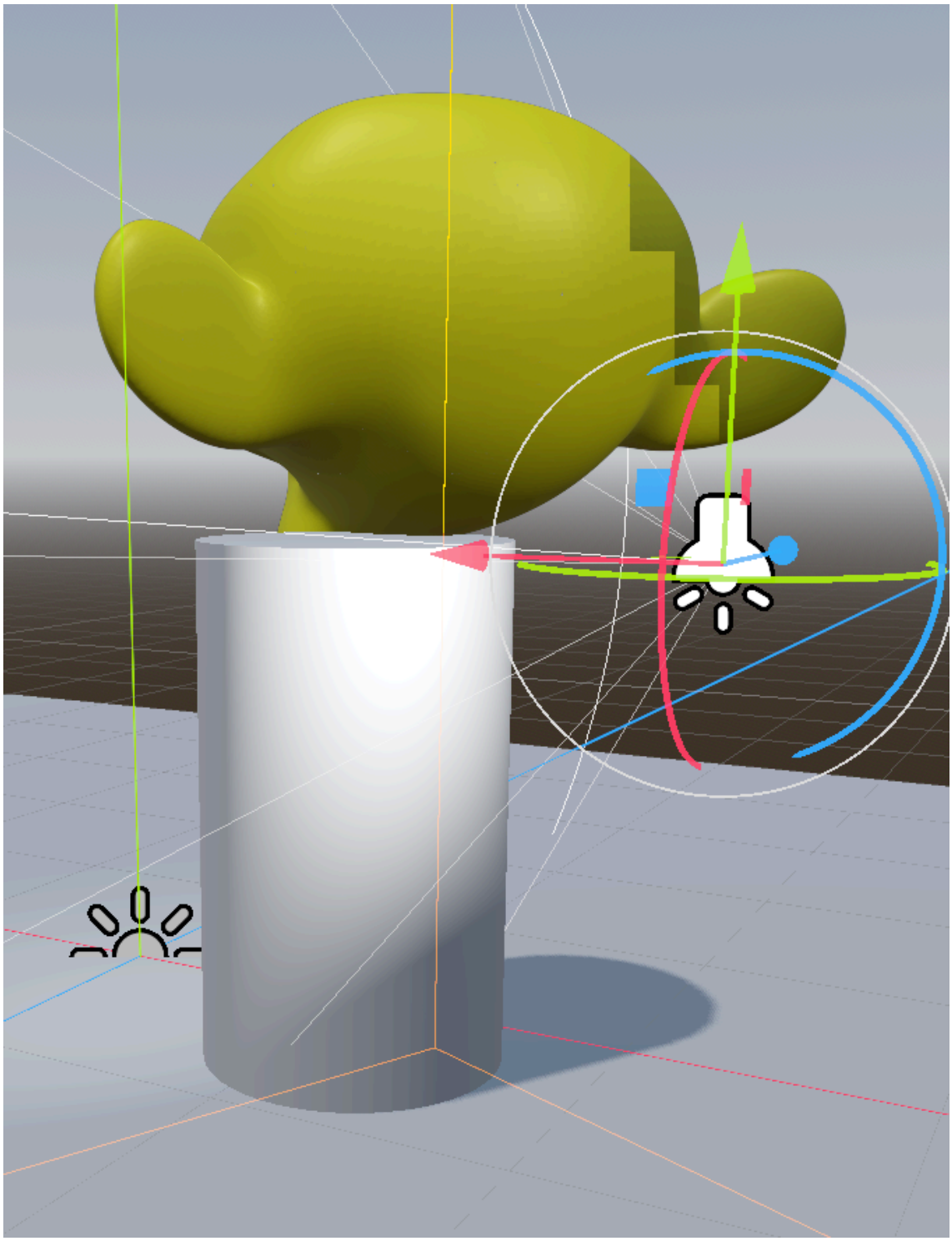
hogy több félig átlátszó splat anyagtulajdonságait átlagoljuk, ezt adjuk át a Godot-nak, és továbbra is realisztikus / használható eredményt kapjunk ebből.

Megjegyzendő, hogy a Mesh2Splat-tal generált modelleknél viszont nem fontos az, hogy az átlátszóságot jól kezeljük, mivel ezek a modellek jellemzően nem sok átlátszóságot tartalmaznak, szimpla modelleknél legalábbis.

Ezen kívül viszont, alapból a 7. ábra jóval jobban belepasszol a jelenetbe, mint a 6. ábra.



7. ábra: Suzanne, a Blender majom megvilágítva Godot által



8. ábra: Suzanne, a Blender majom megvilágítva Godot által egy spotlight-tal is (még bugos)

3.3. Kitérő: Typst

A diplomaterv dokumentumának elkészítéséhez a Typst nevű nyelvet használom, ami egy modern LaTeX megfelelő. Dani nevű szobatársam segítségével konvertáltuk a sablonokat (főleg ő), ez elérhető itt: [10].

3.3.1. Rövid bemutatás

open source, cég háttere, szintaktika, elterjedés

3.3.2. Miért jobb

word/libreoffice-nál, latexnél

Word: kód blokkok, számozás, képek számozása (?), bibliography beszúró GUI izébizénél sokkal egyszerűbb, stb

LaTeX: 20 package a fordításhoz, hosszú build time, csúnya szintaktika, végtelen mennyiségű fájl amik elég nehezen átláthatók, fordításkor is teleszemeteli a dolgot

4. Fejezet

Önálló munka értékelése, eredmények

4.1. When Hamsters Attack TD

játék bemutatása, integrációja a pluginnal, mint példa felhasználás [11]

4.2. Tanszék modelljének példafelhasználása

mire? Legyen valami kis konkrét demo belőle? Itt lehetne demozni, ha a háttér GSplates, míg a WhenHamstersAttackTD-nél, ha a karakterek GSplatesek

4.3. Asset library plugin

Felkerül végre??

4.4. Instancing mérése felhasznált memóriával

4.5. Még mit lehetne értékelésnek, eredménynek?

4.6. További teendők a Dipterv 2-ig (végleges doksiig)

- KÉSZ: Godotos relightolás esetén fixálni a spotlightnál: a 3d pozíció nem jól van beállítva pixelenként
- Relightolható branch (a saját relightolásos) működjön a nem relightolt dolgokkal egyszerre, ennek visszamerge-elése a mainbe, esetleg PR a kínai csávónak (ha nyitott rá)
- GLTF szabvány implementálása, tesztelés vele
- a WhenHamstersAttackTD [11] játék átírása félig (vagy teljesen) GSplatos modellek használatára (esetleg tanszéki terem mint background?, vagy azzal is egy példafelhasználás?)
- ezen dokumentum finomítása, véglegesítése
  - irodalomjegyzék vs lábjegyzet? Melyiket mennyire? Mind mehet irodalomjegyzékbe, nem baj ha nagyon sok?

4.7. TODO a dipterv1 doksi leadásáig!

- kódblokkok számozásának kipróbálása az önlabos doksi szövegében
- írni az ebben a félévben végzett munkáról
  - ehhez pedig számozott képeket beszúrni, kipróbálni a számozást fejezetek között is (melyik számozás legyen (6 vs 4.1, ábrához)? - ennek eldöntése, formázással bajlódás majd ráér később)
  - képaláírások, kép forrásának jelölése hogyan?
- referenciák a képekre, kódblokkokra, más fejezetekre a doksin belül
- bemutató diasor
  - előző diasorból kiindulva, 1-2 diát megtartva
  - a félév során elvégzett lépések, munkák 1-1 dián
  - typst egy dián, sablonról, elkészített dokumentációról szót ejteni
  - utsó dián a 4.6-nál taglaltak felsorolása, konzultálni ezekről a konzulenssel, időpontot kérni ezen dokumentum átnézésére, és finomítására
  - további dolgok időbeosztása a nyáron, konzultációk hogyan, ilyesmi

5. Fejezet

Összefoglaló

Sokat tanultunk a dipterv alatt, megismerkedtünk ezzel, azzal, godot rendering pipeline, gaussian splatting, fájlformátumok, typst, stb

a plugin használható, viszonylag jó integrációval, fenn van (??) az asset store-ban, hozzátettem bizonyos fejlesztéseket, mások meg itt és itt érhetők el, … (külön asset store item a relightolható mesh-esnek??, ha nem merge-eli majd?)

Irodalomjegyzék

[1] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, és George Drettakis, „3D Gaussian Splatting for Real-Time Radiance Field Rendering”, 2023, 2308.04079. [Online]. Elérhető: https://arxiv.org/abs/2308.04079

[2] Stefano Scolari, „Mesh2Splat: Fast mesh to 3D Gaussian splat conversion”. [Online]. Elérhető: https://github.com/electronicarts/mesh2splat

[3] „Khronos Announces glTF Gaussian Splatting Extension”. [Online]. Elérhető: https://www.khronos.org/news/press/gltf-gaussian-splatting-press-release

[4] „Generatív AI és inverz módszerek a képszintézisben”. [Online]. Elérhető: https://vaitkusm.github.io/genai-inverse-graphics-bme-vik-iit/

[5] „GodotGaussianSplatting by 2Retr0”. [Online]. Elérhető: https://github.com/2Retr0/GodotGaussianSplatting

[6] „GodotGaussianSplatting by haztro”. [Online]. Elérhető: https://github.com/haztro/godot-gaussian-splatting

[7] „ReconWorldLab implementációja”. [Online]. Elérhető: https://github.com/ReconWorldLab/godot-gaussian-splatting

[8] „Godot Gaussian Splatting Game”. [Online]. Elérhető: https://github.com/4321ba/GodotGaussianSplattingGame/tree/vitavehicle

[9] „GodotGaussianSplatting issue az AMD bughoz”. [Online]. Elérhető: https://github.com/2Retr0/GodotGaussianSplatting/issues/3

[10] „Typst BME Dipterv Sablon”. [Online]. Elérhető: https://codeberg.org/pilpol/bme-dipterv-sablon

[11] „When Hamsters Attack TD”. [Online]. Elérhető: https://github.com/4321ba/WhenHamstersAttackTD

Függelék

TODO AI nyilatkozat majd ide!